

```
In [107... # DSC 630
# Winter 2024
# Predicting Post Tommy John Performance
# Working Document.
# Created By Matthew Heinen
# 11/28/24
```

```
In [108... # Import packages.
import pandas as pd

#Read in pitcher dataset.
pitcher_df = pd.read_csv("Pitcher_Data_2019_2024.csv")
```

```
In [109... # Verify Load.
pitcher_df.head(10)
```

```
Out[109...      pitches  player_id  player_name  year  total_pitches  pitch_percent  ba  iso  babip  slg  ...  batter_run_value_per_100  xobp  xslg  pitcher_run
```

0	3687	545333	Bauer, Trevor	2019	3687	100	0.230	0.199	0.290	0.429	...	0.063792	0.310	0.402
1	3553	458681	Lynn, Lance	2019	3553	100	0.243	0.146	0.322	0.390	...	-0.860062	0.282	0.372
2	3495	593958	Rodríguez, Eduardo	2019	3495	100	0.253	0.138	0.318	0.391	...	-0.338054	0.295	0.375
3	3448	434378	Verlander, Justin	2019	3448	100	0.172	0.188	0.219	0.361	...	-1.556323	0.233	0.346
4	3384	544931	Strasburg, Stephen	2019	3384	100	0.210	0.139	0.276	0.349	...	-1.086111	0.270	0.337
5	3362	543037	Cole, Gerrit	2019	3362	100	0.186	0.156	0.276	0.343	...	-1.470375	0.232	0.315
6	3358	501985	Minor, Mike	2019	3358	100	0.244	0.151	0.288	0.395	...	-0.660185	0.295	0.389
7	3332	605400	Nola, Aaron	2019	3332	100	0.233	0.160	0.297	0.393	...	-0.603451	0.315	0.394
8	3332	669456	Bieber, Shane	2019	3332	100	0.230	0.163	0.298	0.393	...	-0.765006	0.277	0.417
9	3299	571578	Corbin, Patrick	2019	3299	100	0.227	0.149	0.295	0.375	...	-0.722431	0.294	0.414

10 rows × 71 columns

```
In [110... # Print all column names
print("Column names:")
print(pitcher_df.columns.tolist())
```

Column names:

```
['pitches', 'player_id', 'player_name', 'year', 'total_pitches', 'pitch_percent', 'ba', 'iso', 'babip', 'slg', 'woba', 'xwoba', 'xba', 'hits', 'abs', 'launch_speed', 'launch_angle', 'spin_rate', 'velocity', 'effective_speed', 'whiffs', 'swings', 'takes', 'eff_min_vel', 'release_extension', 'pos3_int_start_distance', 'pos4_int_start_distance', 'pos5_int_start_distance', 'pos6_int_start_distance', 'pos7_int_start_distance', 'pos8_int_start_distance', 'pos9_int_start_distance', 'pitcher_run_exp', 'run_exp', 'bat_speed', 'swing_length', 'pa', 'bip', 'singles', 'doubles', 'triples', 'hrs', 'so', 'k_percent', 'bb', 'bb_percent', 'api_break_z_with_gravity', 'api_break_z_induced', 'api_break_x_arm', 'api_break_x_batter_in', 'hyper_speed', 'bbdist', 'hardhit_percent', 'barrels_per_bbe_percent', 'barrels_per_pa_percent', 'release_pos_z', 'release_pos_x', 'plate_x', 'plate_z', 'obp', 'barrels_total', 'batter_run_value_per_100', 'xobp', 'xslg', 'pitcher_run_value_per_100', 'xbadiff', 'xobpdiff', 'xslgdiff', 'wobadiff', 'swing_miss_percent', 'arm_angle']
```

```
In [111... # trim dataset to relative features for pitcher evaluation
```

```
#Define columns to keep
selected_columns = ['pitches', 'player_name', 'year',
                    'total_pitches', 'pitch_percent', 'ba', 'iso', 'babip',
                    'slg', 'woba', 'xwoba', 'xba', 'hits', 'abs', 'spin_rate', 'velocity', 'effective_speed',
                    'whiffs', 'swings', 'takes', 'eff_min_vel', 'release_extension',
                    'pitcher_run_exp', 'run_exp', 'pa', 'bip', 'singles', 'doubles',
                    'triples', 'hrs', 'so', 'k_percent', 'bb', 'bb_percent',
                    'api_break_z_with_gravity', 'api_break_z_induced', 'api_break_x_arm',
                    'api_break_x_batter_in', 'hyper_speed', 'bbdist', 'hardhit_percent',
                    'barrels_per_bbe_percent', 'barrels_per_pa_percent', 'release_pos_z',
                    'release_pos_x', 'plate_x', 'plate_z', 'obp', 'barrels_total',
                    'batter_run_value_per_100', 'xobp', 'xslg', 'pitcher_run_value_per_100',
                    'xbadiff', 'xobpdiff', 'xslgdiff', 'wobadiff', 'swing_miss_percent',
                    'arm_angle']
```

```
# Filter dataframe to only relevant pitching stats
pitcher_df = pitcher_df[selected_columns]
```

```
# Preview Dataframe.
pitcher_df.head(3)
```

```
Out[111...      pitches  player_name  year  total_pitches  pitch_percent  ba  iso  babip  slg  woba  ...  batter_run_value_per_100  xobp  xslg  pitcher_run_va
```

0	3687	Bauer, Trevor	2019	3687	100	0.230	0.199	0.290	0.429	0.316	...	0.063792	0.310	0.402
1	3553	Lynn, Lance	2019	3553	100	0.243	0.146	0.322	0.390	0.294	...	-0.860062	0.282	0.372
2	3495	Rodríguez, Eduardo	2019	3495	100	0.253	0.138	0.318	0.391	0.307	...	-0.338054	0.295	0.375

3 rows × 59 columns

```
In [112... # Find columns with missing values.
# Count the number of missing values per column
missing_values = pitcher_df.isnull().sum()

# Filter columns with missing values
columns_with_missing = missing_values[missing_values > 0]

# Print the column names and missing values
print("Columns with missing values and their counts:")
print(columns_with_missing)
```

Columns with missing values and their counts:

```
ba          1
iso          1
babip       10
slg          1
xwoba        1
xba          10
spin_rate    2
velocity      1
effective_speed  1
eff_min_vel   1
release_extension  1
api_break_z_with_gravity  1
api_break_z_induced  1
api_break_x_arm  1
api_break_x_batter_in  1
hyper_speed   6
bbdist        6
hardhit_percent  9
barrels_per_bbe_percent  9
release_pos_z  1
release_pos_x  1
plate_x       1
plate_z       1
xobp          7
xslg         10
xbadiff       10
xobpdiff       7
xslgdiff      10
wobadiff       1
swing_miss_percent  1
arm_angle     845
dtype: int64
```

```
In [113... # Replace the missing values in the arm_angle column with the median arm angle.
# Arm angle is the only column with significant amount of missing values.

# Calculate the median of the 'arm_angle' column
median_arm_angle = pitcher_df['arm_angle'].median()

# Replace missing values (NaN) with the median
pitcher_df['arm_angle'].fillna(median_arm_angle, inplace=True)
```

Acquire Injury data

```
In [115... # Tommy John Data Acquired from
# https://medium.com/analytics-vidhya/lets-model-using-random-forest-to-predict-ucl-reconstruction-6185ebdc853e

# Read in Tommy John surgery data list
#Read in pitcher dataset.
tj_df = pd.read_csv("TJ_Surgery_List.csv")

# Preview
tj_df.head(5)
```

```
Out[115...
      Player  TJ  Team Level Position Throws Country High School College(s) Age ... IP K-BB% ERA- FIP- G.1 GS.1 IP.1 K-BB%.1 ERA-.1 FI
      Surgery Date
0  Braxton Garrett  12/1/24  MIA  AAA      P      L  United States  Alabama      NaN  26.0 ... NaN NaN NaN NaN NaN NaN NaN NaN
1  Wen-Hui Pan  11/1/24  PHI  A+      P      R  Taiwan      NaN      NaN  21.0 ... NaN NaN NaN NaN NaN NaN NaN NaN
2  Julian Aguiar  10/11/24  CIN  MLB      P      R  United States  California  Cypress  23.0 ... NaN NaN NaN NaN NaN NaN NaN NaN
3  Joe Musgrove  10/11/24  SD  MLB      P      R  United States  California      NaN  31.0 ... NaN NaN NaN NaN NaN NaN NaN NaN
4  Cade Feeney  10/1/24  BOS  A+      P      R  United States  North Dakota  North Dakota State  22.0 ... NaN NaN NaN NaN NaN NaN NaN NaN
```

5 rows x 42 columns

```
In [116... # Print all column names
print("Column names in the DataFrame:")
print(tj_df.columns.tolist())
```

Column names in the DataFrame:
['Player', 'TJ Surgery Date', 'Team', 'Level', 'Position', 'Throws', 'Country', 'High School', 'College(s)', 'Age', 'Return Date (same level)', 'Recovery Time (months)', 'mlbamid', 'fgid', 'Surgeon(s)', 'Post-TJ MLB G', 'Post-TJ MLB IP/PA', 'Active', 'Year', 'Month', 'Day', 'Started\nThrowing', 'Mound', 'Bullpen', 'Live\nHitters', 'Game', 'Setback', 'Setback Type', 'Setback.1', 'Setback Type.1', 'G', 'GS', 'IP', 'K-BB%', 'ERA-', 'FIP-', 'G.1', 'GS.1', 'IP.1', 'K-BB%.1', 'ERA-.1', 'FIP-.1']

```
In [117... # trim dataset to only include information about the tj injury an surgery.

#Define columns to keep
selected_columns = ['Player', 'TJ Surgery Date', 'Team', 'Level', 'Position', 'Throws',
                    'Country', 'High School', 'College(s)', 'Age', 'Return Date (same level)',
                    'Recovery Time (months)', 'Surgeon(s)']

# Filter dataframe to only relevant pitching stats
tj_df = tj_df[selected_columns]

# Preview Dataframe.
tj_df.head(3)
```

Out[117...

	Player	TJ Surgery Date	Team	Level	Position	Throws	Country	High School	College(s)	Age	Return Date (same level)	Recovery Time (months)	Surgeon(s)
0	Braxton Garrett	12/1/24	MIA	AAA	P	L	United States	Alabama	NaN	26.0	NaN	NaN	NaN
1	Wen-Hui Pan	11/1/24	PHI	A+	P	R	Taiwan	NaN	NaN	21.0	NaN	NaN	NaN
2	Julian Aguiar	10/11/24	CIN	MLB	P	R	United States	California	Cypress	23.0	NaN	NaN	Dr. Keith Meister

Feature engineer the player name columns to match with both datasets to merge into one.

```
In [119... # Format the name column on pitcher_df dataframe.
# Convert 'name' column from 'LastName, FirstName' to 'FirstName LastName'
# Use string split to split on the comma.
pitcher_df['player_name'] = pitcher_df['player_name'].apply(lambda x: ' '.join(x.split(', ')[::-1]))

# Preview
pitcher_df.head(3)
```

Out[119...

	pitches	player_name	year	total_pitches	pitch_percent	ba	iso	babip	slg	woba	...	batter_run_value_per_100	xobp	xslg	pitcher_run_va
0	3687	Trevor Bauer	2019	3687	100	0.230	0.199	0.290	0.429	0.316	...	0.063792	0.310	0.402	
1	3553	Lance Lynn	2019	3553	100	0.243	0.146	0.322	0.390	0.294	...	-0.860062	0.282	0.372	
2	3495	Eduardo Rodriguez	2019	3495	100	0.253	0.138	0.318	0.391	0.307	...	-0.338054	0.295	0.375	

3 rows x 59 columns

Use Inner Join to Merge DataFrames.

```
In [121... # Perform an inner join on 'Player_name' form pitcher_df and 'Player' from tj_fd
# Use inner Join
merged_df = pd.merge(pitcher_df, tj_df, left_on='player_name', right_on='Player', how='inner')

# Preview new merged df
merged_df.head(10)
```

Out [121...

	pitches	player_name	year	total_pitches	pitch_percent	ba	iso	babip	slg	woba	...	Level	Position	Throws	Country	High School	College(s)
0	3553	Lance Lynn	2019	3553	100	0.243	0.146	0.322	0.390	0.294	...	MLB	P	R	United States	Indiana	Mississippi
1	3167	Lance Lynn	2023	3167	100	0.260	0.225	0.292	0.485	0.348	...	MLB	P	R	United States	Indiana	Mississippi
2	2608	Lance Lynn	2021	2608	100	0.209	0.129	0.265	0.338	0.264	...	MLB	P	R	United States	Indiana	Mississippi
3	2039	Lance Lynn	2024	2039	100	0.247	0.155	0.288	0.402	0.314	...	MLB	P	R	United States	Indiana	Mississippi
4	2022	Lance Lynn	2022	2022	100	0.247	0.160	0.294	0.407	0.304	...	MLB	P	R	United States	Indiana	Mississippi
5	1408	Lance Lynn	2020	1408	100	0.206	0.180	0.243	0.386	0.287	...	MLB	P	R	United States	Indiana	Mississippi
6	3448	Justin Verlander	2019	3448	100	0.172	0.188	0.219	0.361	0.243	...	MLB	P	R	United States	Virginia	Old Dominion
7	2607	Justin Verlander	2022	2607	100	0.186	0.085	0.240	0.270	0.222	...	MLB	P	R	United States	Virginia	Old Dominion
8	2602	Justin Verlander	2023	2602	100	0.226	0.143	0.265	0.369	0.284	...	MLB	P	R	United States	Virginia	Old Dominion
9	1580	Justin Verlander	2024	1580	100	0.274	0.176	0.303	0.450	0.337	...	MLB	P	R	United States	Virginia	Old Dominion

10 rows × 72 columns

Above we see that we have successfully merged while maintaining separate records for each season pitched.

```
In [123... # Find shape of df
merged_df.shape
```

Out [123... (1845, 72)

Create Variable to Denote if a Season was Pre or Post Tommy John Surgery.

```
In [125... # Find the data type for year (season).
# This will inform how we format the TJ return Year.

# Find the data type of the 'age' column
column_dtype = merged_df['year'].dtype

# Print the data type
print(f"The data type of 'age' column is: {column_dtype}")
```

The data type of 'age' column is: int64

```
In [126... # Rename year as season to limit confusion in the naming schema.
merged_df.rename(columns={'year': 'season'}, inplace=True)
```

```
In [127... # Find columns with missing values.
# Count the number of missing values per column
missing_values = merged_df.isnull().sum()

# Filter columns with missing values
columns_with_missing = missing_values[missing_values > 0]

# Print the column names and missing values
print("Columns with missing values and their counts:")
print(columns_with_missing)
```

```
Columns with missing values and their counts:
High School      291
College(s)       707
Return Date (same level)  227
Recovery Time (months)  547
Surgeon(s)       730
dtype: int64
```

Here we see that we are actually missing some return dates. This is likely because those player have yet to return from Tommy John. Seeing as we are trying to predict post Tommy John performance these records will be dropped from the df.

```
In [129... # Drop players who have not yet returned from tj surgery.
# use drop na to drop all records with missing values in the 'Return Date (same level)' column.
merged_df = merged_df.dropna(subset=['Return Date (same level)'])
```

```
In [130... # Suppress the warning globally
pd.options.mode.chained_assignment = None
```

```
In [131... # Add a return date year
# Convert 'date' column to datetime and extract the year as an integer
```

```
# cast as int to
merged_df['tj_return_year'] = pd.to_datetime(merged_df['Return Date (same level)'], format='%m/%d/%y').dt.year.astype(int)
```

```
In [132... # Create a post tj surgery column to denote if a season was after the recovery from a tommy john surgery.
# value is True if 'season' >= 'tj_return_year'
merged_df['post_tj'] = merged_df['season'] >= merged_df['tj_return_year']

# Preview
merged_df.head(5)
```

Out[132...

	pitches	player_name	season	total_pitches	pitch_percent	ba	iso	babip	slg	woba	...	Throws	Country	High School	College(s)	Age	Return Date (same level)
0	3553	Lance Lynn	2019	3553	100	0.243	0.146	0.322	0.390	0.294	...	R	United States	Indiana	Mississippi	28.0	4/6/17
1	3167	Lance Lynn	2023	3167	100	0.260	0.225	0.292	0.485	0.348	...	R	United States	Indiana	Mississippi	28.0	4/6/17
2	2608	Lance Lynn	2021	2608	100	0.209	0.129	0.265	0.338	0.264	...	R	United States	Indiana	Mississippi	28.0	4/6/17
3	2039	Lance Lynn	2024	2039	100	0.247	0.155	0.288	0.402	0.314	...	R	United States	Indiana	Mississippi	28.0	4/6/17
4	2022	Lance Lynn	2022	2022	100	0.247	0.160	0.294	0.407	0.304	...	R	United States	Indiana	Mississippi	28.0	4/6/17

5 rows x 74 columns

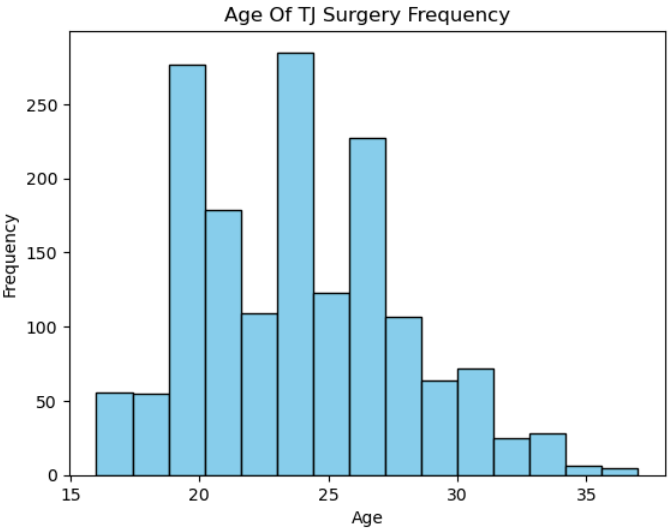
EDA

```
In [134... # Build Histogram showing age distribuion of tj Surgery
import pandas as pd
import matplotlib.pyplot as plt

# Plotting the histogram of the 'age' column
plt.hist(merged_df['Age'], bins=15, color='skyblue', edgecolor='black')

# Adding labels and title
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.title('Age Of TJ Surgery Frequency')

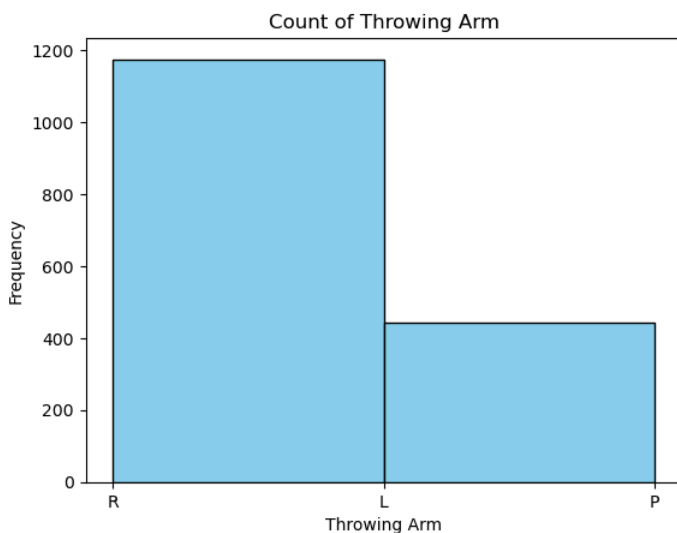
# Show the plot
plt.show()
```



```
In [135... # Plotting the histogram of the 'age' column
plt.hist(merged_df['Throws'], bins=2, color='skyblue', edgecolor='black')

# Adding labels and title
plt.xlabel('Throwing Arm')
plt.ylabel('Frequency')
plt.title('Count of Throwing Arm')

# Show the plot
plt.show()
```

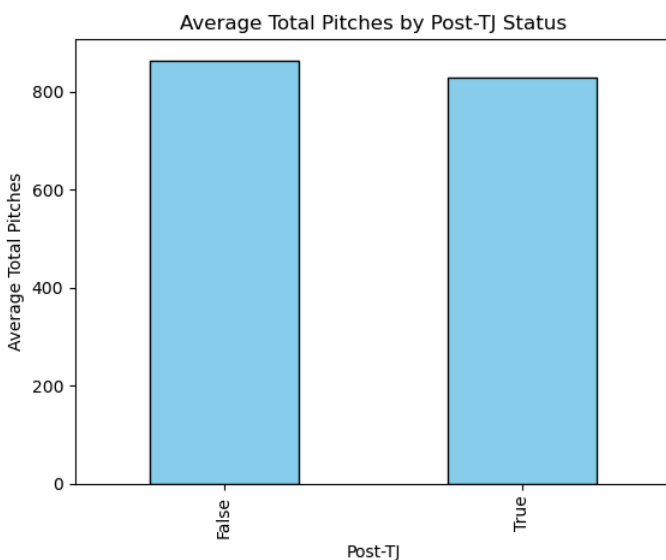


```
In [136... # Build a bar graph showing total pitches pre and post TJ
# Group by 'post_tj' and calculate the average of 'total_pitches'
avg_pitches = merged_df.groupby('post_tj')['total_pitches'].mean()

# Plot the bar graph
avg_pitches.plot(kind='bar', color='skyblue', edgecolor='black')

# Adding labels and title
plt.xlabel('Post-TJ')
plt.ylabel('Average Total Pitches')
plt.title('Average Total Pitches by Post-TJ Status')

# Show the plot
plt.show()
```



```
In [137... # Test for statistically significant difference in average pitches between the two groups .
# Import Packages.
import pandas as pd
import scipy.stats as stats

# Separate the data into two groups based on 'post_tj' column
group_true = merged_df[merged_df['post_tj'] == True]['total_pitches']
group_false = merged_df[merged_df['post_tj'] == False]['total_pitches']

# Perform the independent two-sample t-test
t_stat, p_value = stats.ttest_ind(group_true, group_false)

# Print the t-statistic and p-value
print("T-statistic:", t_stat)
print("P-value:", p_value)

# Interpret the result
# Significance level 95%
alpha = 0.05
if p_value < alpha:
    print("There is a statistically significant difference between the groups (reject H0).")
```

```

else:
    print("There is no statistically significant difference between the groups (fail to reject H0).")

```

T-statistic: -0.5806736179588687

P-value: 0.561541410794217

There is no statistically significant difference between the groups (fail to reject H0).

```

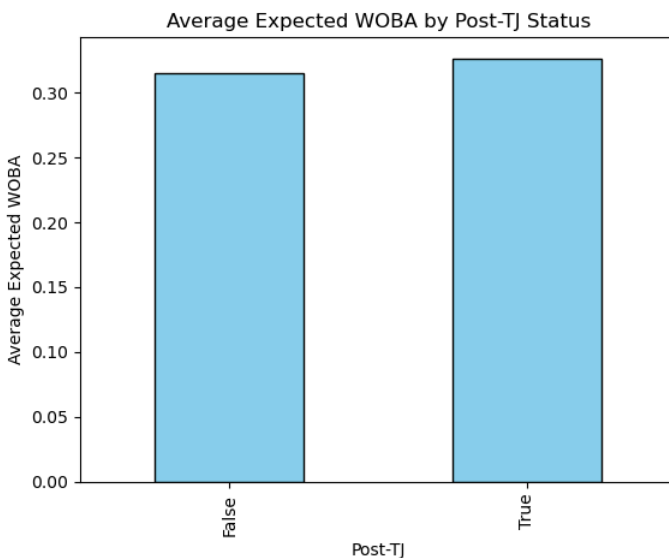
In [138... # Build bargraph showing xwoba by pre and post TJ surgery.
# Group by 'post_tj' and calculate the average of 'xwoba'
avg_xwoba = merged_df.groupby('post_tj')['xwoba'].mean()

# Plot the bar graph
avg_xwoba.plot(kind='bar', color='skyblue', edgecolor='black')

# Adding labels and title
plt.xlabel('Post-TJ')
plt.ylabel('Average Expected WOBA')
plt.title('Average Expected WOBA by Post-TJ Status')

# Show the plot
plt.show()

```



```

In [139... # Test for statistically significant difference in XWOBA between the two groups .
# Import Packages.
import pandas as pd
import scipy.stats as stats

# Separate the data into two groups based on 'post_tj' column
group_true = merged_df[merged_df['post_tj'] == True]['xwoba']
group_false = merged_df[merged_df['post_tj'] == False]['xwoba']

# Perform the independent two-sample t-test
t_stat, p_value = stats.ttest_ind(group_true, group_false)

# Print the t-statistic and p-value
print("T-statistic:", t_stat)
print("P-value:", p_value)

# Interpret the result
# Significance level 95%
alpha = 0.05
if p_value < alpha:
    print("There is a statistically significant difference between the groups (reject H0).")
else:
    print("There is no statistically significant difference between the groups (fail to reject H0).")

```

T-statistic: 2.5563327355234486

P-value: 0.010668942813686981

There is a statistically significant difference between the groups (reject H0).

```

In [140... # Dataframe Shape and size.

```

```
merged_df.shape
```

```
Out[140... (1618, 74)
```

```

In [141... # Count the number of unique player names
unique_player_count = merged_df['player_name'].nunique()

# Print the result
print(f"Number of unique player names: {unique_player_count}")

```

Number of unique player names: 483

```

In [142... # Find the number players in the data set where we have performance data
# from before and after their TJ surgery.

```

```
# Group by 'player_name' and check if both True and False exist for 'post_tj'
players_with_both = merged_df.groupby('player_name')['post_tj'].nunique()

# Filter for players who have both True and False
players_with_both = players_with_both[players_with_both == 2]

# Count the number of such players
unique_player_count = players_with_both.count()

# Print the result
print(f"Number of unique player names with both True and False in post_tj: {unique_player_count}")
```

Number of unique player names with both True and False in post_tj: 91

Milestone 4

```
In [144... # Preview DF
merged_df.head(10)
```

Out [144...

	pitches	player_name	season	total_pitches	pitch_percent	ba	iso	babip	slg	woba	...	Throws	Country	High School	College(s)	Age	Return Date (same level)
0	3553	Lance Lynn	2019	3553	100	0.243	0.146	0.322	0.390	0.294	...	R	United States	Indiana	Mississippi	28.0	4/6/17
1	3167	Lance Lynn	2023	3167	100	0.260	0.225	0.292	0.485	0.348	...	R	United States	Indiana	Mississippi	28.0	4/6/17
2	2608	Lance Lynn	2021	2608	100	0.209	0.129	0.265	0.338	0.264	...	R	United States	Indiana	Mississippi	28.0	4/6/17
3	2039	Lance Lynn	2024	2039	100	0.247	0.155	0.288	0.402	0.314	...	R	United States	Indiana	Mississippi	28.0	4/6/17
4	2022	Lance Lynn	2022	2022	100	0.247	0.160	0.294	0.407	0.304	...	R	United States	Indiana	Mississippi	28.0	4/6/17
5	1408	Lance Lynn	2020	1408	100	0.206	0.180	0.243	0.386	0.287	...	R	United States	Indiana	Mississippi	28.0	4/6/17
6	3448	Justin Verlander	2019	3448	100	0.172	0.188	0.219	0.361	0.243	...	R	United States	Virginia	Old Dominion	37.0	4/9/22
7	2607	Justin Verlander	2022	2607	100	0.186	0.085	0.240	0.270	0.222	...	R	United States	Virginia	Old Dominion	37.0	4/9/22
8	2602	Justin Verlander	2023	2602	100	0.226	0.143	0.265	0.369	0.284	...	R	United States	Virginia	Old Dominion	37.0	4/9/22
9	1580	Justin Verlander	2024	1580	100	0.274	0.176	0.303	0.450	0.337	...	R	United States	Virginia	Old Dominion	37.0	4/9/22

10 rows x 74 columns

```
In [145... # List all column names
# this will help us identify how to summarize pitchers pre and post TJ surgery performance.
print(merged_df.columns)

Index(['pitches', 'player_name', 'season', 'total_pitches', 'pitch_percent',
      'ba', 'iso', 'babip', 'slg', 'woba', 'xwoba', 'xba', 'hits', 'abs',
      'spin_rate', 'velocity', 'effective_speed', 'whiffs', 'swings', 'takes',
      'eff_min_vel', 'release_extension', 'pitcher_run_exp', 'run_exp', 'pa',
      'bip', 'singles', 'doubles', 'triples', 'hrs', 'so', 'k_percent', 'bb',
      'bb_percent', 'api_break_z_with_gravity', 'api_break_z_induced',
      'api_break_x_arm', 'api_break_x_batter_in', 'hyper_speed', 'bbdist',
      'hardhit_percent', 'barrels_per_bbe_percent', 'barrels_per_pa_percent',
      'release_pos_z', 'release_pos_x', 'plate_x', 'plate_z', 'obp',
      'barrels_total', 'batter_run_value_per_100', 'xobp', 'xslg',
      'pitcher_run_value_per_100', 'xbadiff', 'xobpdiff', 'xslgdiff',
      'wobadiff', 'swing_miss_percent', 'arm_angle', 'Player',
      'TJ Surgery Date', 'Team', 'Level', 'Position', 'Throws', 'Country',
      'High School', 'College(s)', 'Age', 'Return Date (same level)',
      'Recovery Time (months)', 'Surgeon(s)', 'tj_return_year', 'post_tj'],
      dtype='object')

In [146... # Drop duplicate or unnecessary columns.
merged_df = merged_df.drop(columns=['total_pitches', 'pitch_percent', 'Player'])

In [147... # Create a subset df for only post TJ surgery performance.
post_tj_df = merged_df[merged_df['post_tj'] == True]

# Create a subset df for only pre TJ surgery performance
pre_tj_df = merged_df[merged_df['post_tj'] == False]

In [148... # Preview post TJ dataframes.
post_tj_df.head(3)
```


Out [148...

	pitches	player_name	season	ba	iso	babip	slg	woba	xwoba	xba	...	Throws	Country	High School	College(s)	Age	Return Date (same level)	Recovery Time (months)	Surg
0	3553	Lance Lynn	2019	0.243	0.146	0.322	0.390	0.294	0.283	0.222	...	R	United States	Indiana	Mississippi	28.0	4/6/17	17.0	Dr. i
1	3167	Lance Lynn	2023	0.260	0.225	0.292	0.485	0.348	0.337	0.252	...	R	United States	Indiana	Mississippi	28.0	4/6/17	17.0	Dr. i
2	2608	Lance Lynn	2021	0.209	0.129	0.265	0.338	0.264	0.256	0.198	...	R	United States	Indiana	Mississippi	28.0	4/6/17	17.0	Dr. i

3 rows × 71 columns

In [149...

```
# Preview pre TJ dataframes.  
pre_tj_df.head(3)
```

Out [149...

	pitches	player_name	season	ba	iso	babip	slg	woba	xwoba	xba	...	Throws	Country	High School	College(s)	Age	Return Date (same level)	Recovery Time (months)	Sur
6	3448	Justin Verlander	2019	0.172	0.188	0.219	0.361	0.243	0.248	0.186	...	R	United States	Virginia	Old Dominion	37.0	4/9/22	18.0	
10	73	Justin Verlander	2020	0.150	0.300	0.091	0.450	0.264	0.259	0.161	...	R	United States	Virginia	Old Dominion	37.0	4/9/22	18.0	
27	3297	Jacob deGrom	2019	0.207	0.117	0.285	0.323	0.250	0.250	0.201	...	R	United States	Florida	Stetson	35.0	9/13/24	15.0	

3 rows × 71 columns

In [150...

```
# Define aggregate functions for all features of the data set.  
# Each column will have an agg function  
agg_funcs = {  
    # Take the mean of the following features.  
    'pitches': 'mean',  
    'ba': 'mean',  
    'iso': 'mean',  
    'babip': 'mean',  
    'slg': 'mean',  
    'woba': 'mean',  
    'xwoba': 'mean',  
    'xba': 'mean',  
    'hits': 'mean',  
    'abs': 'mean',  
    'spin_rate': 'mean',  
    'velocity': 'mean',  
    'effective_speed': 'mean',  
    'whiffs': 'mean',  
    'swings': 'mean',  
    'takes': 'mean',  
    'eff_min_vel': 'mean',  
    'release_extension': 'mean',  
    'pitcher_run_exp': 'mean',  
    'run_exp': 'mean',  
    'pa': 'mean',  
    'bip': 'mean',  
    'singles': 'mean',  
    'doubles': 'mean',  
    'triples': 'mean',  
    'hrs': 'mean',  
    'so': 'mean',  
    'k_percent': 'mean',  
    'bb': 'mean',  
    'bb_percent': 'mean',  
    'api_break_z_with_gravity': 'mean',  
    'api_break_z_induced': 'mean',  
    'api_break_x_arm': 'mean',  
    'api_break_x_batter_in': 'mean',  
    'hyper_speed': 'mean',  
    'bbdist': 'mean',  
    'hardhit_percent': 'mean',  
    'barrels_per_bbe_percent': 'mean',  
    'barrels_per_pa_percent': 'mean',  
    'release_pos_z': 'mean',  
    'release_pos_x': 'mean',  
    'plate_x': 'mean',  
    'plate_z': 'mean',  
    'obp': 'mean',  
    'barrels_total': 'mean',  
    'batter_run_value_per_100': 'mean',  
    'xobp': 'mean',  
    'xslg': 'mean',  
    'pitcher_run_value_per_100': 'mean',  
    'xbadiff': 'mean',  
    'xobpdiff': 'mean',  
    'xslgdiff': 'mean',  
    'wobadiff': 'mean',  
    'swing_miss_percent': 'mean',  
}
```

```
'arm_angle': 'mean',

# Take the count of the following features.
'season': 'count',

# Take the mode of the following features.
# In theory all of these values should be the same anyway
'TJ Surgery Date': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Team': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Level': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Position': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Throws': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Country': lambda x: x.mode()[0] if not x.mode().empty else None ,
'High School': lambda x: x.mode()[0] if not x.mode().empty else None ,
'College(s)': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Age': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Return Date (same level)': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Recovery Time (months)': lambda x: x.mode()[0] if not x.mode().empty else None ,
'Surgeon(s)': lambda x: x.mode()[0] if not x.mode().empty else None ,
'tj_return_year': lambda x: x.mode()[0] if not x.mode().empty else None ,
'post_tj': lambda x: x.mode()[0] if not x.mode().empty else None
}
```

```
In [151... # Use group by to apply aggregate functions to the post_tj_df.
# Group on player name
# reset index to keep group by variable as column.
post_tj_df = post_tj_df.groupby('player_name').agg(agg_funcs).reset_index()

# Preview
post_tj_df.head(5)
```

Out[151...

	player_name	pitches	ba	iso	babip	slg	woba	xwoba	xba	hits	...	Throws	Country	High School	College(s)
0	A.J. Ladwig	41.000000	0.400000	0.533000	0.3080	0.9330	0.562000	0.599000	0.438000	6.000000	...	R	United States	Nebraska	Wichita State
1	A.J. Minter	759.333333	0.226000	0.130667	0.3030	0.3565	0.284333	0.280833	0.215500	37.333333	...	L	United States	Texas	Texas A&M
2	A.J. Puk	728.000000	0.240200	0.122400	0.3224	0.3626	0.298200	0.273200	0.213000	37.200000	...	L	United States	Iowa	Florida
3	Aaron Barrett	49.500000	0.375000	0.125000	0.4000	0.5000	0.455000	0.365500	0.244500	3.500000	...	R	United States	Indiana	Mississippi, Wabash Valley
4	Aaron Bummer	781.000000	0.223167	0.066667	0.3170	0.2900	0.274333	0.270000	0.208167	39.000000	...	L	United States	Arizona	Nebraska

5 rows x 71 columns

```
In [152... # Validate the season counting stat.
post_tj_df[['season']].head(5)
```

Out[152...

	season
0	1
1	6
2	5
3	2
4	6

```
In [153... # Use group by to apply aggregate functions to the pre_tj_df.
# Group on player name
# reset index to keep group by variable as column.
pre_tj_df = pre_tj_df.groupby('player_name').agg(agg_funcs).reset_index()

# Preview
pre_tj_df.head(5)
```

Out [153]...

	player_name	pitches	ba	iso	babip	slg	woba	xwoba	xba	hits	...	Throws	Country	High School	Colle
0	Adrian Morejon	185.333333	0.3100	0.290333	0.338667	0.599667	0.400667	0.371667	0.288333	13.333333	...	L	Cuba	None	
1	Andrew Kittredge	537.000000	0.2385	0.135000	0.278000	0.373750	0.281500	0.300500	0.254750	32.250000	...	R	United States	Washington	Washi
2	Anthony Bender	623.500000	0.2285	0.137500	0.281000	0.366000	0.296500	0.304000	0.228500	31.000000	...	R	United States	California	Santa
3	Anthony Gose	252.000000	0.1440	0.141000	0.199000	0.285000	0.237500	0.264500	0.174000	8.500000	...	L	United States	California	
4	Antonio Senzatela	1483.800000	0.2906	0.198600	0.301200	0.489400	0.353600	0.364400	0.298200	110.000000	...	R	Venezuela	None	

5 rows x 71 columns

```
In [154]... # Validate the season counting stat.
pre_tj_df[[ 'season' ]].head(5)
```

	season
0	3
1	4
2	2
3	2
4	5

```
In [155]... # Rename the target variables xwoba and pitches in the post_tj_df.
# This is done before adding them back to the same dataset.
post_tj_df = post_tj_df.rename(columns={'xwoba': 'post_tj_xwoba',
                                         'pitches': 'post_tj_pitches'})
```

```
In [156]... # Merge dataframes back together to build models.
# only include the target metrics from the post_tj_df.
# Perform inner join on player_name

model_df = pre_tj_df.merge(post_tj_df[['player_name', 'post_tj_xwoba', 'post_tj_pitches']],
                           on='player_name', how='inner')
# Preview df
model_df.head(5)
```

Out [156]...

	player_name	pitches	ba	iso	babip	slg	woba	xwoba	xba	hits	...	High School	College(s)	Age	Return Date (same level)
0	Adrian Morejon	185.333333	0.3100	0.290333	0.338667	0.599667	0.400667	0.371667	0.288333	13.333333	...	None	None	22.0	6/7/22
1	Andrew Kittredge	537.000000	0.2385	0.135000	0.278000	0.373750	0.281500	0.300500	0.254750	32.250000	...	Washington	Washington	32.0	8/18/23
2	Anthony Bender	623.500000	0.2285	0.137500	0.281000	0.366000	0.296500	0.304000	0.228500	31.000000	...	California	Santa Rosa	27.0	3/28/24
3	Anthony Gose	252.000000	0.1440	0.141000	0.199000	0.285000	0.237500	0.264500	0.174000	8.500000	...	California	None	31.0	8/7/24
4	Antonio Senzatela	1483.800000	0.2906	0.198600	0.301200	0.489400	0.353600	0.364400	0.298200	110.000000	...	None	None	28.0	9/16/24

5 rows x 73 columns

```
In [157]... # Drop more unnecessary columns.
# Date info is no longer necessary after identifying pre and post TJ stats.
model_df = model_df.drop(columns=['post_tj', 'TJ Surgery Date', 'Return Date (same level)'])
```

```
In [158]... # Create a copy of model_df with no dummy variables.
model_df_no_dummies = model_df

# build dummy variables for categorical variables.
# drop first to avoid multi colinearity.
model_df = pd.get_dummies(model_df, columns=['Team', 'Level', 'Position', 'Throws', 'Country',
                                             'High School', 'College(s)', 'Surgeon(s)'], drop_first=True)
# Preview
model_df.head(5)
```

Out [158...

	player_name	pitches	ba	iso	babip	slg	woba	xwoba	xbt	hits	...	College(s)_Yavapai	Surgeon(s)_Dr. Christopher Camp	s
0	Adrian Morejon	185.333333	0.3100	0.290333	0.338667	0.599667	0.400667	0.371667	0.288333	13.333333	...	False	False	
1	Andrew Kittredge	537.000000	0.2385	0.135000	0.278000	0.373750	0.281500	0.300500	0.254750	32.250000	...	False	False	
2	Anthony Bender	623.500000	0.2285	0.137500	0.281000	0.366000	0.296500	0.304000	0.228500	31.000000	...	False	False	
3	Anthony Gose	252.000000	0.1440	0.141000	0.199000	0.285000	0.237500	0.264500	0.174000	8.500000	...	False	False	
4	Antonio Senzatela	1483.800000	0.2906	0.198600	0.301200	0.489400	0.353600	0.364400	0.298200	110.000000	...	False	False	

5 rows x 183 columns

In [159...

```
# Remove the categorical feature data from the model_df_no_dummies
model_df_no_dummies = model_df_no_dummies.drop(columns=['Team', 'Level', 'Position', 'Throws', 'Country',
                                                         'High School', 'College(s)', 'Surgeon(s)'])
```

In [160...

```
# Find non-numeric columns in the DataFrame
non_numeric_cols = model_df.select_dtypes(exclude=['number']).columns

# Loop through df and convert the catagorical boolean to intigers (0 or 1).
for col in non_numeric_cols:
    model_df[col] = pd.factorize(model_df[col])[0] # Convert string categories to integers

# Preview df
model_df.head(5)
```

Out [160...

	player_name	pitches	ba	iso	babip	slg	woba	xwoba	xbt	hits	...	College(s)_Yavapai	Surgeon(s)_Dr. Christopher Camp	s
0	0	185.333333	0.3100	0.290333	0.338667	0.599667	0.400667	0.371667	0.288333	13.333333	...	0	0	
1	1	537.000000	0.2385	0.135000	0.278000	0.373750	0.281500	0.300500	0.254750	32.250000	...	0	0	
2	2	623.500000	0.2285	0.137500	0.281000	0.366000	0.296500	0.304000	0.228500	31.000000	...	0	0	
3	3	252.000000	0.1440	0.141000	0.199000	0.285000	0.237500	0.264500	0.174000	8.500000	...	0	0	
4	4	1483.800000	0.2906	0.198600	0.301200	0.489400	0.353600	0.364400	0.298200	110.000000	...	0	0	

5 rows x 183 columns

Build Backward Stepwise Regression

P-value Threshold = 0.05

In [162...

```
# Create a train test split.

# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split

# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['player_name', 'post_tj_xwoba', 'post_tj_pitches'])
target = model_df['post_tj_xwoba']

# Split into 80 % training 20% test
# use train_test_split

features_train, features_test, target_train, target_test= train_test_split(features,
                                                                            target, test_size=0.2, random_state=42)
```

In [163...

```
# Build stepwise regression with a p value threshold of 0.05

# Load Libraries
import pandas as pd
import statsmodels.api as sm
import warnings
import numpy as np

# Suppress Warning
warnings.simplefilter("ignore", category=RuntimeWarning)

# Add a constant to the model (intercept)
features_train = sm.add_constant(features_train)

# Fit the initial model
model = sm.OLS(target_train, features_train).fit()

# Backward stepwise regression
while True:
    # Get the p-values of the model
```

```

p_values = model.pvalues
# Find the maximum p-value
max_p_value = p_values.max()

# If the maximum p-value is greater than 0.05, remove that variable
if max_p_value > 0.05:
    # Identify the variable to remove
    variable_to_remove = p_values.idxmax()
    # Remove the variable from the model
    features_train = features_train.drop(variable_to_remove, axis=1)
    # Remove from test data as well
    features_test = features_test.drop(variable_to_remove, axis=1)
    # Refit the model
    model = sm.OLS(target_train, features_train).fit()
else:
    break

# Print the summary of the final model # Commented out for length, Includes too many features
#print("Final model summary:")
#print(model.summary())

# Prepare the test set for prediction
# Add constant to test set
features_test_final = features_test.copy()
features_test_final = sm.add_constant(features_test)

```

```

In [164... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = model.predict(features_train)
test_predictions = model.predict(features_test_final)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print(" P-Value Threshold 0.05, Including Demographic Info")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")

P-Value Threshold 0.05, Including Demographic Info
Training R²: 1.0, Training RMSE: 2.707345606265507e-14
Test R²: -5.043677829308281, Test RMSE: 0.1051648416000168

```

Rebuild the stepwise model using PCA and a Standard Scaler

```

In [166... # Create a train test split.

# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split

# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['player_name', 'post_tj_xwoba', 'post_tj_pitches'])
target = model_df['post_tj_xwoba']

# Split into 80 % training 20% test
# use train_test_split

features_train, features_test, target_train, target_test = train_test_split(features,
                                                                            target, test_size=0.2, random_state=42)

```

```

In [219... # Load Libraries
import pandas as pd
import statsmodels.api as sm
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import warnings
import numpy as np

# Suppress Warning
warnings.simplefilter("ignore", category=RuntimeWarning)

# Define Standard Scaler
scaler = StandardScaler()

# Standardize the features before applying PCA (important for PCA)
features_train_scaled = scaler.fit_transform(features_train)
features_test_scaled = scaler.transform(features_test)

# Step 2: Apply PCA to reduce dimensionality

```

```
pca = PCA() # You can specify the number of components you want, e.g., PCA(n_components=5)
features_train_pca = pca.fit_transform(features_train_scaled)
features_test_pca = pca.transform(features_test_scaled)

# Step 3: Add a constant to the PCA transformed features (for the intercept)
features_train_pca = sm.add_constant(features_train_pca)
features_test_pca = sm.add_constant(features_test_pca)

# Fit the initial model
model = sm.OLS(target_train, features_train_pca).fit()

# Backward stepwise regression
while True:
    # Get the p-values of the model
    p_values = model.pvalues
    # Find the maximum p-value
    max_p_value = p_values.max()

    # If the maximum p-value is greater than 0.05, remove that variable
    if max_p_value > 0.05:
        # Identify the variable to remove
        variable_to_remove = p_values.idxmax()
        # Remove the variable from the model
        features_train_pca = features_train_pca.drop(variable_to_remove, axis=1)
        # Remove from test data as well
        features_test_pca = features_test_pca.drop(variable_to_remove, axis=1)
        # Refit the model
        model = sm.OLS(target_train, features_train_pca).fit()
    else:
        break

# Print the summary of the final model # Commented out for length
#print("Final model summary:")
#print(model.summary())

# Prepare the test set for prediction
# Add constant to test set
features_test_final = features_test_pca.copy()
features_test_final = sm.add_constant(features_test_pca)
```

```
In [899... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = model.predict(features_train_pca)
test_predictions = model.predict(features_test_final)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print(" P-Value Threshold 0.05")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")
```

P-Value Threshold 0.05
 Training R²: 1.0, Training RMSE: 1.052131994376454e-15
 Test R²: -2.7837209935171416, Test RMSE: 0.08321075675275957

Build Recursive Feature Elimination

```
In [ ]: # Load libraries
from sklearn.feature_selection import RFECV
from sklearn import linear_model

# create linear regression
ols = linear_model.LinearRegression()

# Recursively Eliminate Features
rfecv = RFECV(estimator=ols, step=1, cv=5, scoring="neg_mean_squared_error")
rfecv.fit(features_train, target_train)
rfecv.transform(features_train)
```

```
In [734... # number of best features
rfecv.n_features_
```

```
Out[734... 1
```

```
In [736... # Get the selected features
selected_features = features_train.columns[rfecv.support_]

# Output the selected features
print("Selected Features:", selected_features.tolist())
```

Selected Features: ['Country_Cuba']

```
In [738... # Get coefficients
coefficients = rfecv.estimator_.coef_

# Print coefficients and feature names in a readable format
print("Coefficients and corresponding feature names:")
for feature, coef in zip(selected_features, coefficients):
    print(f"{feature}: {coef:.4f}")

# Get the intercept of the model
intercept = rfecv.estimator_.intercept_
print("Model intercept: ", intercept)
```

Coefficients and corresponding feature names:
Country_Cuba: 0.1562
Model intercept: 0.16899999999999993

```
In [742... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = rfecv.predict(features_train)
test_predictions = rfecv.predict(features_test_final)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print("Recursive Feature Elimination")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")
```

Recursive Feature Elimination
Training R²: 0.11960125016478218, Training RMSE: 0.04958080013424756
Test R²: -1.0768630633544118, Test RMSE: 0.061648689734219664

Rebuild stepwise linear regression without the categorical or demographic data.

```
In [935... # Create a train test split.

# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split

# create features
# Drop target variables and unnecessary columns
features = model_df_no_dummies.drop(columns=['player_name', 'post_tj_xwoba', 'post_tj_pitches'])
target = model_df_no_dummies['post_tj_xwoba']

# Split into 80 % training 20% test
# use train_test_split

features_train, features_test, target_train, target_test = train_test_split(features,
                                                                            target, test_size=0.2, random_state=42)
```

```
In [937... # Build stepwise regression with a p value threshold of 0.05

# Load Libraries
import pandas as pd
import statsmodels.api as sm

# Add a constant to the model (intercept)
features_train = sm.add_constant(features_train)

# Fit the initial model
model = sm.OLS(target_train, features_train).fit()

# Backward stepwise regression
while True:
    # Get the p-values of the model
    p_values = model.pvalues
    # Find the maximum p-value
    max_p_value = p_values.max()

    # If the maximum p-value is greater than 0.05, remove that variable
    if max_p_value > 0.05:
        # Identify the variable to remove
        variable_to_remove = p_values.idxmax()
        # Remove the variable from the model
        features_train = features_train.drop(variable_to_remove, axis=1)
        # Remove from test data as well
        features_test = features_test.drop(variable_to_remove, axis=1)
        # Refit the model
```

```

        model = sm.OLS(target_train, features_train).fit()
    else:
        break

# Print the summary of the final model
print("Final model summary:")
print(model.summary())

# Prepare the test set for prediction
# Add constant to test set
features_test_final = features_test.copy()
features_test_final = sm.add_constant(features_test)

```

Final model summary:

```

                        OLS Regression Results
=====
Dep. Variable:          post_tj_xwoba      R-squared:                0.329
Model:                  OLS              Adj. R-squared:           0.256
Method:                 Least Squares     F-statistic:                4.484
Date:                   Sun, 09 Feb 2025   Prob (F-statistic):         0.000411
Time:                   17:39:36          Log-Likelihood:             123.92
No. Observations:       72               AIC:                       -231.8
Df Residuals:           64               BIC:                       -213.6
Df Model:                7
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	0.5357	0.196	2.733	0.008	0.144	0.927
pitches	0.0375	0.014	2.608	0.011	0.009	0.066
velocity	-0.7997	0.247	-3.243	0.002	-1.292	-0.307
effective_speed	0.7975	0.246	3.240	0.002	0.306	1.289
swings	-0.0378	0.014	-2.617	0.011	-0.067	-0.009
takes	-0.0373	0.014	-2.599	0.012	-0.066	-0.009
eff_min_vel	-0.8059	0.245	-3.295	0.002	-1.295	-0.317
plate_x	-0.0883	0.033	-2.639	0.010	-0.155	-0.021

```

=====
Omnibus:                 3.835   Durbin-Watson:           1.817
Prob(Omnibus):            0.147   Jarque-Bera (JB):         3.634
Skew:                     0.226   Prob(JB):                  0.162
Kurtosis:                 4.003   Cond. No.:                 9.17e+04
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 9.17e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```

In [939... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = model.predict(features_train)
test_predictions = model.predict(features_test_final)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
print(" P-Value Threshold 0.05, Stepwise Model No Demographic")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")

P-Value Threshold 0.05, Stepwise Model No Demographic
Training R²: 0.329076157714872, Training RMSE: 0.0432823104003254
Test R²: -0.3643743285618508, Test RMSE: 0.04996739275732819

```

```

In [907... model_df.shape

```

```

Out[907... (91, 183)

```

```

In [ ]: # REBuild Recursive Feature Elimination without Dummy Variables

```

```

In [100... # Create a train test split.

# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split

# create features
# Drop target variables and unnecessary columns
features = model_df_no_dummies.drop(columns=['player_name', 'post_tj_xwoba', 'post_tj_pitches'])
target = model_df_no_dummies['post_tj_xwoba']

# Split into 80 % training 20% test
# use train_test_split

```



```
features_train, features_test, target_train, target_test= train_test_split(features,
                                                                           target, test_size=0.2, random_state=42)
```

```
In [101... # Load libraries
from sklearn.feature_selection import RFECV
from sklearn import linear_model

# create linear regression
ols = linear_model.LinearRegression()

# Recursively Eliminate Features
rfecv = RFECV(estimator=ols, step=1, cv =5, scoring="neg_mean_squared_error")
rfecv.fit(features_train, target_train)
rfecv.transform(features_train)
```

```
Out[101... array([[ 0.256   ,  0.154   ,  0.315   , ...,  0.023   ,
         0.045   ,  0.021   ],
       [ 0.181   ,  0.192   ,  0.238   , ...,  0.011   ,
         0.043   ,  0.018   ],
       [ 0.299   ,  0.44    ,  0.22433333, ...,  0.03166667,
         0.106   ,  0.04033333],
       ...,
       [ 0.232   ,  0.127   ,  0.306   , ...,  0.023   ,
         0.035   ,  0.021   ],
       [ 0.2195  ,  0.2175  ,  0.239   , ..., -0.028   ,
        -0.0095  , -0.017   ],
       [ 0.2385  ,  0.089   ,  0.328   , ...,  0.0115  ,
        -0.001   ,  0.006   ]])
```

```
In [101... # number of best features
rfecv.n_features_
```

```
Out[101... 16
```

```
In [101... # Get the selected features
selected_features = features_train.columns[rfecv.support_]

# Output the selected features
print("Selected Features:", selected_features.tolist())
```

Selected Features: ['ba', 'iso', 'babip', 'slg', 'xwoba', 'xba', 'velocity', 'effective_speed', 'eff_min_vel', 'obp', 'xobp', 'xslg', 'xbadiff', 'xobpdiff', 'xslgdiff', 'wobadiff']

```
In [101... # Get coefficients
coefficients = rfecv.estimator_.coef_

# Print coefficients and feature names in a readable format
print("Coefficients and corresponding feature names:")
for feature, coef in zip(selected_features, coefficients):
    print(f"{feature}: {coef:.4f}")

# Get the intercept of the model
intercept = rfecv.estimator_.intercept_
print("Model intercept: ", intercept)
```

Coefficients and corresponding feature names:
ba: 7.2499
iso: 10.4884
babip: -0.3885
slg: -6.8033
xwoba: -0.2945
xba: 3.5951
velocity: -0.7391
effective_speed: 0.7393
eff_min_vel: -0.7524
obp: 0.3708
xobp: -0.0708
xslg: -3.7440
xbadiff: 3.6548
xobpdiff: 0.4417
xslgdiff: -3.0593
wobadiff: -1.8432
Model intercept: 0.3582879853402157

```
In [933... # Evaluate Fit

# Load Libraries
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

# Predictions
train_predictions = rfecv.predict(features_train)
test_predictions = rfecv.predict(features_test)

# R² Calculation
r2_train = r2_score(target_train, train_predictions)
r2_test = r2_score(target_test, test_predictions)

# RMSE Calculation
rmse_train = np.sqrt(mean_squared_error(target_train, train_predictions))
rmse_test = np.sqrt(mean_squared_error(target_test, test_predictions))

# Output the results
```

```
print("Recursive Feature Elimination No Dummies")
print(f"Training R²: {r2_train}, Training RMSE: {rmse_train}")
print(f"Test R²: {r2_test}, Test RMSE: {rmse_test}")
```

Recursive Feature Elimination No Dummies
 Training R²: 0.31060471199784656, Training RMSE: 0.04387407546154057
 Test R²: -0.16992362960771001, Test RMSE: 0.04626991223683221

Build Classification model predicting if a pitcher will through certain number of pitches.

```
In [942... # Determine pitching threshold.
# Find the median number of pitches from the original pitcher df.
# This includes all pitchers in the era not just those who have had TJ surgery.
```

```
median_pitches = pitcher_df['pitches'].median()
print("Median of pitches:", median_pitches)
```

Median of pitches: 507.0

```
In [946... # Lets set the threshold at 500 pitches to say they returned and were sucessful/ effective.
# Create binary predictor for post TJ pitches.
# Use greater than or equal to 500 as the threshold
# Create a dummy variable based on the condition
# Build copy of original df
model_df_class = model_df.copy()
model_df_class['Eff_Pitch_count'] = (model_df_class['post_tj_pitches'] >= 500).astype(int)

# Preview
model_df.head(5)
```

```
Out[946...
   player_name  pitches    ba    iso    babip    slg    woba    xwoba    xba    hits  ...  Surgeon(s)_Dr.  Surgeon(s)_Dr.  Surg
   player_name  pitches    ba    iso    babip    slg    woba    xwoba    xba    hits  ...  Surgeon(s)_Dr.  Surgeon(s)_Dr.  Surg
0           0    185.333333  0.3100  0.290333  0.338667  0.599667  0.400667  0.371667  0.288333  13.333333  ...      0           0
1           1    537.000000  0.2385  0.135000  0.278000  0.373750  0.281500  0.300500  0.254750  32.250000  ...      0           0
2           2    623.500000  0.2285  0.137500  0.281000  0.366000  0.296500  0.304000  0.228500  31.000000  ...      0           0
3           3    252.000000  0.1440  0.141000  0.199000  0.285000  0.237500  0.264500  0.174000   8.500000  ...      0           0
4           4   1483.800000  0.2906  0.198600  0.301200  0.489400  0.353600  0.364400  0.298200 110.000000  ...      0           0
```

5 rows x 184 columns

Train the classification model

```
In [103... # Create a train test split.

# Import packages.
import pandas as pd
from sklearn.model_selection import train_test_split

# create features
# Drop target variables and unnecessary columns
features = model_df.drop(columns=['player_name', 'post_tj_xwoba', 'post_tj_pitches', 'Eff_Pitch_count'])
target = model_df['Eff_Pitch_count']

# Split into 80 % training 20% test
# use train_test_split

features_train, features_test, target_train, target_test = train_test_split(features,
                                                                            target, test_size=0.2, random_state=42)
```

```
In [103... # Load libraries
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
import warnings

# Suppress future warnings
warnings.filterwarnings("ignore", category=FutureWarning)

# Initialize the Min-Max scaler
mmScaler = MinMaxScaler()

# define pipeline
pipe = Pipeline([("MinMaxScaler", mmScaler), ("classifier", RandomForestClassifier())])

# Create dictionary with candidate learning algorithms and their hyperparameters
# Build a logistic regression, random forrest, and decesion tree
search_space = [
```

```

# Logistic Regression param
{"classifier": [LogisticRegression(max_iter=500, solver='liblinear')],
 "classifier__penalty": ['l1', 'l2'],
 "classifier__C": np.logspace(0, 4, 10)},
#Random Forest Param
{"classifier": [RandomForestClassifier()],
 "classifier__n_estimators": [10, 100, 1000],
 "classifier__max_features": [1, 2, 3]},
# Decesion Tree Param
{"classifier": [DecisionTreeClassifier()],
 # Allow for unlimited depth
 "classifier__max_depth": [None, 2, 4, 6, 8, 10, 12, 20],
 "classifier__min_samples_split": [2, 5, 10],
 "classifier__min_samples_leaf": [1, 2, 4],
 "classifier__criterion": ['gini', 'entropy']},
# Bagging parm
{"classifier": [BaggingClassifier(base_estimator=DecisionTreeClassifier(), random_state=42)],
 "classifier__n_estimators": [10, 50, 100]},
# Boosting decesion tree param
{"classifier": [AdaBoostClassifier(base_estimator=DecisionTreeClassifier(max_depth=1), random_state=42)],
 "classifier__n_estimators": [50, 100, 300],
 "classifier__learning_rate": [0.01, 0.1, 1.0]},
# Gradient Boosting param
{"classifier": [GradientBoostingClassifier(random_state=42)],
 "classifier__n_estimators": [50, 100, 300],
 "classifier__learning_rate": [0.01, 0.1, 0.2],
 "classifier__max_depth": [3, 5, 7]},]

# create grid search
gridsearch = GridSearchCV(pipe, search_space, cv=5, verbose=0)

# Fit gridsearch
best_model = gridsearch.fit(features_train, target_train)

# Print Best model
print("The best model is: ", best_model.best_estimator_)

The best model is: Pipeline(steps=[('MinMaxScaler', MinMaxScaler()),
 ('classifier',
  GradientBoostingClassifier(learning_rate=0.2, n_estimators=50,
                             random_state=42))])

```

```

In [103... #Find the best estimator
best_estimator = best_model.best_estimator_

# Get the feature names from df
feature_names = features_train.columns

# Find the importances from the gradient boost classifier
importances = best_estimator.named_steps['classifier'].feature_importances_

# Create a DataFrame of the importances and sort by importance
feature_importance = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
# only display the top 10 features of importance.
feature_importance = feature_importance.sort_values(by='Importance', ascending=False).head(10)

# Print
print("Gradient Boosting Feature Importances (Sorted):")
print(feature_importance)

```

```

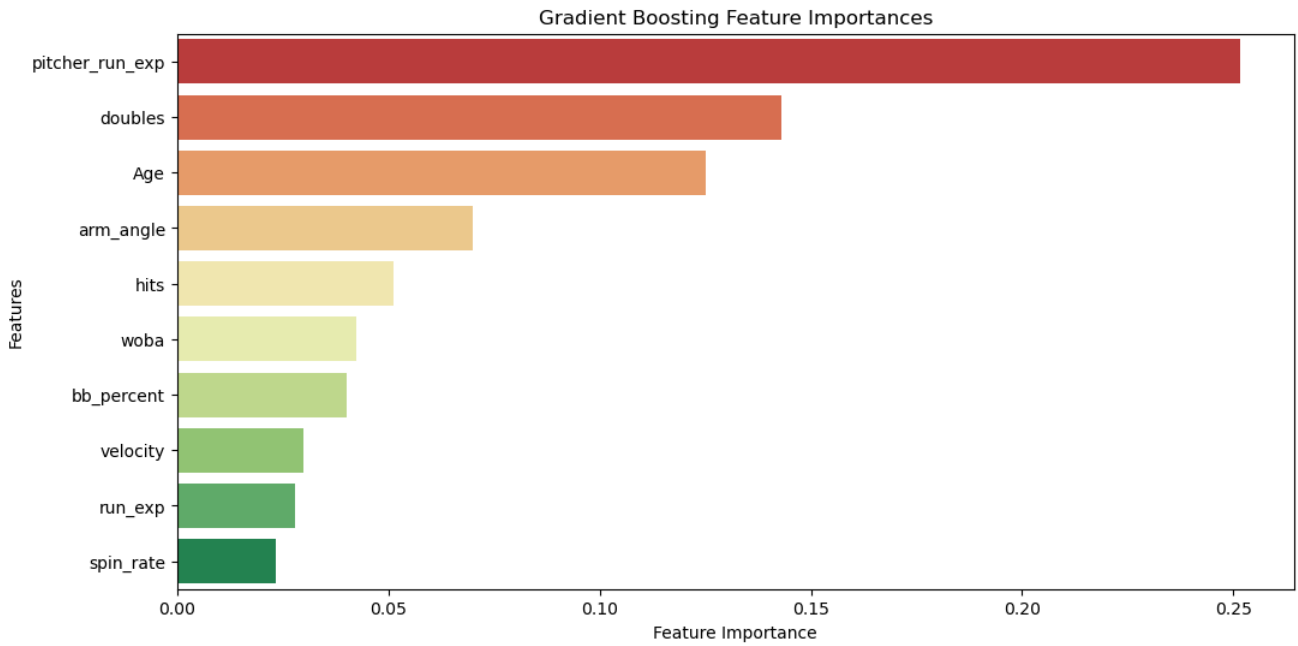
Gradient Boosting Feature Importances (Sorted):
   Feature  Importance
18  pitcher_run_exp    0.251756
23    doubles         0.142996
56      Age          0.125123
54  arm_angle        0.069909
8      hits          0.051224
5      woba          0.042361
29  bb_percent        0.040115
11  velocity         0.030013
19    run_exp         0.027851
10  spin_rate         0.023461

```

```

In [103... import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
# Plot
plt.figure(figsize=(12, 6))
sns.barplot(x=feature_importance['Importance'], y=feature_importance['Feature'], palette="RdYlGn")
plt.xlabel("Feature Importance")
plt.ylabel("Features")
plt.title("Gradient Boosting Feature Importances")
plt.show()

```



```
In [104... # Find model accuracy
# find the accuracy of this model on the training set.

accuracy = best_model.score(features_train, target_train)
print(f"Accuracy of the best model and hyperparameters found in the grid search Training Data: {accuracy:.2f}")

# find the accuracy of this model on the test set.

accuracy = best_model.score(features_test, target_test)
print(f"Accuracy of the best model and hyperparameters found in the grid search Test Data: {accuracy:.2f}")
```

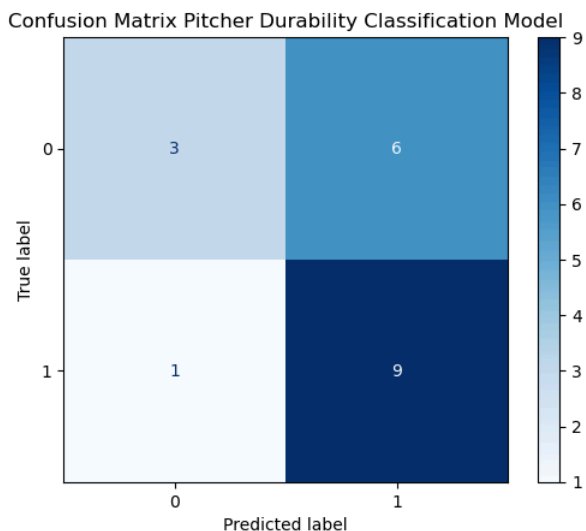
Accuracy of the best model and hyperparameters found in the grid search Training Data: 1.00
Accuracy of the best model and hyperparameters found in the grid search Test Data: 0.63

```
In [105... # Import packages
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Make predictions on the test set
predictions = best_estimator.predict(features_test)

# Create confusion matrix
cm = confusion_matrix(target_test, predictions)

# Display the confusion matrix using ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=best_estimator.classes_)
disp.plot(cmap='Blues') # You can change the color map to 'Reds', 'Greens', etc.
plt.title("Confusion Matrix Pitcher Durability Classification Model")
plt.show()
```



In []: